Title:	A Unified Executors Proposal for C++
Authors:	Jared Hoberock, jhoberock@nvidia.com Michael Garland, mgarland@nvidia.com Chris Kohlhoff, chris@kohlhoff.com Chris Mysen, mysen@google.com Carter Edwards, hcedwar@sandia.gov Gordon Brown, gordon@codeplay.com
Other Contributors:	Hans Boehm, hboehm@google.com Thomas Heller, thom.heller@gmail.com Lee Howes, lwh@fb.com Bryce Lelbach, brycelelbach@gmail.com Hartmut Kaiser, hartmut.kaiser@gmail.com Bryce Lelbach, brycelelbach@gmail.com Gor Nishanov, gorn@microsoft.com Thomas Rodgers, rodgert@twrodgers.com David Hollman, dshollm@sandia.gov Michael Wong, michael@codeplay.com
Document Number:	P0443R7
Date:	2018-05-07
Audience:	SG1 - Concurrency and Parallelism, LEWG
Reply-to:	sg1-exec@googlegroups.com
Abstract:	This paper proposes a programming model for executors, which are modular components for creating execution. The design of this proposal is described in paper P0761.

0.1 Changelog

0.1.1 Revision 7

Revision 7 of this proposal corrects wording bugs discovered by the authors after Revision 6â€™s publication.

- Enhanced `static_query_v` to result in a default property value for executors which do not provide a `query` function for the property of interest
- Revise `then_execute` and `bulk_then_execute`â€™s operational semantics to allow user functions to handle incoming exceptions thrown by preceding execution agents
- Introduce `exception_arg` to disambiguate the user functionâ€™s exceptional overload from its nonexceptional overload in `then_execute` and `bulk_then_execute`

0.1.2 Revision 6

Revision 6 of this proposal corrects bugs and omissions discovered by the authors after Revision 5â€™s publication, and introduces an enhancement improving the safety of the design.

- Enforce mutual exclusion of behavioral properties via the type system instead of via convention
- Introduce missing `execution::require` adaptations
- Allow executors to opt-out of invoking factory functions when appropriate
- Various bug fixes and corrections

0.1.3 Revision 5

Revision 5 of this proposal responds to feedback requested during the 2017 Albuquerque ISO C++ Standards Committee meeting and introduces changes which allow properties to better interoperate with polymorphic executor wrappers and also simplify `execution::require`â€™s behavior.

- Defined general property type requirements
- Elaborated specification of standard property types
- Simplified `execution::require`â€™s specification
- Enhanced polymorphic executor wrapper
  - Templatized `execution::executor<SupportableProperties...>`
  - Introduced `prefer_only` property adaptor
- Responded to Albuquerque feedback
  - From SG1
    - Execution contexts are now optional properties of executors

        - Eliminated ill-specified caller-agent forward progress properties
        - Elaborated `Future`â€™s requirements to incorporate forward progress
        - Reworded operational semantics of execution functions to use similar language as the blocking properties
        - Elaborated `static_thread_pool`â€™s specification to guarantee that threads in the bool boost-block their work
        - Elaborated operational semantics of execution functions to note that forward progress guarantees are specific to the concrete executor type
    - From LEWG
        - Eliminated named `BaseExecutor` concept
        - Simplified general executor requirements
        - Enhanced the `OneWayExecutor` introductory paragraph
        - Eliminated `has_*_member` type traits
- Minor changes
    - Renamed TS namespace from `concurrency_v2` to `executors_v1`
    - Introduced `static_query_v` enabling static queries
    - Eliminated unused `property_value` trait
    - Eliminated the names `allocator_wrapper_t` and `default_allocator`

0.1.4 Revision 4

- Specified the guarantees implied by `bulk_sequenced_execution`, `bulk_parallel_execution`, and `bulk_unsequenced_execution`

0.1.5 Revision 3

- Introduced `execution::query()` for executor property introspection
- Simplified the design of `execution::prefer()`
- `oneway`, `twoway`, `single`, and `bulk` are now `require()`-only properties
- Introduced properties allowing executors to opt into adaptations that add blocking semantics
- Introduced properties describing the forward progress relationship between caller and agents
- Various minor improvements to existing functionality based on prototyping

0.1.6 Revision 2

- Separated wording from explanatory prose, now contained in paper P0761
- Applied the simplification proposed by paper P0688

0.1.7 Revision 1

- Executor category simplification
- Specified executor customization points in detail
- Introduced new fine-grained executor type traits
    - Detectors for execution functions
    - Traits for introspecting cross-cutting concerns
        - Introspection of mapping of agents to threads
        - Introspection of execution function blocking behavior
- Allocator support for single agent execution functions
- Renamed `thread_pool` to `static_thread_pool`
- New introduction

0.1.8 Revision 0

- Initial design

1 Proposed Wording

1.0.1 Header `<exception>` synopsis

```
namespace std {
namespace experimental {
inline namespace executors_v1 {

  // Exception argument tag
  struct exception_arg_t { explicit exception_arg_t() = default; };
  inline constexpr exception_arg_t exception_arg{};

}
}
}
```

1.1 Exception argument tag

The `exception_arg_t` struct is an empty structure type used as a unique type to disambiguate constructor and function overloading. Specifically, functions passed to `then_execute` and `bulk_then_execute` may have `exception_arg_t` as an argument, immediately followed by an argument that should be interpreted as an exception thrown from a preceding function invocation.

1.1.1 Header `<execution>` synopsis

```
namespace std {
namespace experimental {
inline namespace executors_v1 {
namespace execution {

  // Customization points:

  namespace {
    constexpr unspecified require = unspecified;
    constexpr unspecified prefer = unspecified;
    constexpr unspecified query = unspecified;
  }

  // Customization point type traits:

  template<class Executor, class... Properties> struct can_require;
  template<class Executor, class... Properties> struct can_prefer;
  template<class Executor, class Property> struct can_query;

  template<class Executor, class... Properties>
    constexpr bool can_require_v = can_require<Executor, Properties...>::value;
  template<class Executor, class... Properties>
    constexpr bool can_prefer_v = can_prefer<Executor, Properties...>::value;
  template<class Executor, class Property>
    constexpr bool can_query_v = can_query<Executor, Property>::value;

  // Associated execution context property:

  struct context_t;

  constexpr context_t context;

  // Directionality properties:

  struct oneway_t;
  struct twoway_t;
  struct then_t;

  constexpr oneway_t oneway;
  constexpr twoway_t twoway;
  constexpr then_t then;

  // Cardinality properties:

  struct single_t;
  struct bulk_t;

  constexpr single_t single;
  constexpr bulk_t bulk;

  // Blocking properties:

  struct blocking_t;

  constexpr blocking_t blocking;

  // Properties to allow adaptation of blocking and directionality:

  struct blocking_adaptation_t;

  constexpr blocking_adaptation_t blocking_adaptation;

  // Properties to indicate if submitted tasks represent continuations:

  struct relationship_t;

  constexpr relationship_t relationship;

  // Properties to indicate likely task submission in the future:

  struct outstanding_work_t;

  constexpr outstanding_work_t outstanding_work;
```

```
  // Properties for bulk execution guarantees:

  struct bulk_guarantee_t;

  constexpr bulk_guarantee_t bulk_guarantee;

  // Properties for mapping of execution on to threads:

  struct mapping_t;

  constexpr mapping_t mapping;

  // Memory allocation properties:

  template <typename ProtoAllocator>
  struct allocator_t;

  constexpr allocator_t allocator;

  // Executor type traits:

  template<class Executor> struct is_oneway_executor;
  template<class Executor> struct is_twoway_executor;
  template<class Executor> struct is_then_executor;
  template<class Executor> struct is_bulk_oneway_executor;
  template<class Executor> struct is_bulk_twoway_executor;
  template<class Executor> struct is_bulk_then_executor;

  template<class Executor> constexpr bool is_oneway_executor_v = is_oneway_executor<Executor>::value;
  template<class Executor> constexpr bool is_twoway_executor_v = is_twoway_executor<Executor>::value;
  template<class Executor> constexpr bool is_then_executor_v = is_then_executor<Executor>::value;
  template<class Executor> constexpr bool is_bulk_oneway_executor_v = is_bulk_oneway_executor<Executor>::value;
  template<class Executor> constexpr bool is_bulk_twoway_executor_v = is_bulk_twoway_executor<Executor>::value;
  template<class Executor> constexpr bool is_bulk_then_executor_v = is_bulk_then_executor<Executor>::value;

  template<class Executor, class T> struct executor_future;
  template<class Executor> struct executor_shape;
  template<class Executor> struct executor_index;

  template<class Executor, class T> using executor_future_t = typename executor_future<Executor, T>::type;
  template<class Executor> using executor_shape_t = typename executor_shape<Executor>::type;
  template<class Executor> using executor_index_t = typename executor_index<Executor>::type;

  // Polymorphic executor wrappers:

  class bad_executor;

  template <class... SupportableProperties>
  class executor;
  template<class Property> struct prefer_only;

} // namespace execution
} // inline namespace executors_v1
} // namespace experimental
} // namespace std
```

1.2 Requirements

1.2.1 Customization point objects

(The following text has been adapted from the draft Ranges Technical Specification.)

A *customization point object* is a function object (C++ Std, [function.objects]) with a literal class type that interacts with user-defined types while enforcing semantic requirements on that interaction.

The type of a customization point object shall satisfy the requirements of `CopyConstructible` (C++Std [copyconstructible]) and `Destructible` (C++Std [destructible]).

All instances of a specific customization point object type shall be equal.

Let `t` be a (possibly const) customization point object of type `T`, and `args...` be a parameter pack expansion of some parameter pack `Args...`. The customization point object `t` shall be callable as `t(args...)` when the types of `Args...` meet the requirements specified in that customization point objectâ€™s definition. Otherwise, `T` shall not have a function call operator that participates in overload resolution.

Each customization point object type constrains its return type to satisfy some particular type requirements.

The library defines several named customization point objects. In every translation unit where such a name is defined, it shall refer to the same instance of the customization point object.

[*Note:* Many of the customization points objects in the library evaluate function call expressions with an unqualified name which results

in a call to a user-defined function found by argument dependent name lookup (C++Std [basic.lookup.argdep]). To preclude such an expression resulting in a call to unconstrained functions with the same name in namespace `std`, customization point objects specify that lookup for these expressions is performed in a context that includes deleted overloads matching the signatures of overloads defined in namespace `std`. When the deleted overloads are viable, user-defined overloads must be more specialized (C++Std [temp.func.order]) to be used by a customization point object. *â€“end note*]

1.2.2 `Future` requirements

A type `F` meets the `Future` requirements for some value type `T` if `F` is `std::experimental::future<T>` (defined in the C++ Concurrency TS, ISO/IEC TS 19571:2016). [*Note:* This concept is included as a placeholder to be elaborated, with the expectation that the elaborated requirements for `Future` will expand the applicability of some executor customization points. *â€“end note*]

Forward progress guarantees are a property of the concrete `Future` type. [*Note:* `std::experimental::future<T>::wait()` blocks with forward progress guarantee delegation until the shared state is ready. *â€“end note*]

1.2.3 `ProtoAllocator` requirements

A type `A` meets the `ProtoAllocator` requirements if `A` is `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `allocator_traits<A>::rebind_alloc<U>` meets the allocator requirements (C++Std [allocator.requirements]), where `U` is an object type. [*Note:* For example, `std::allocator<void>` meets the proto-allocator requirements but not the allocator requirements. *â€“end note*] No comparison operator, copy operation, move operation, or swap operation on these types shall exit via an exception.

1.2.4 General requirements on executors

An executor type shall satisfy the requirements of `CopyConstructible` (C++Std [copyconstructible]), `Destructible` (C++Std [destructible]), and `EqualityComparable` (C++Std [equalitycomparable]).

None of these conceptsâ€™ operations, nor an executor typeâ€™s swap operations, shall exit via an exception.

None of these conceptsâ€™ operations, nor an executor typeâ€™s associated execution functions, associated query functions, or other member functions defined in executor type requirements, shall introduce data races as a result of concurrent calls to those functions from different threads.

For any two (possibly const) values `x1` and `x2` of some executor type `X`, `x1 == x2` shall return `true` only if `x1.query(p) == x2.query(p)` for every property `p` where both `x1.query(p)` and `x2.query(p)` are well-formed and result in a non-void type that is `EqualityComparable` (C++Std [equalitycomparable]). [*Note:* The above requirements imply that `x1 == x2` returns `true` if `x1` and `x2` can be interchanged with identical effects. An executor may conceptually contain additional properties which are not exposed by a named property type that can be observed via `execution::query`; in this case, it is up to the concrete executor implementation to decide if these properties affect equality. Returning `false` does not necessarily imply that the effects are not identical. *â€“end note*]

An executor typeâ€™s destructor shall not block pending completion of the submitted function objects. [*Note:* The ability to wait for completion of submitted function objects may be provided by the associated execution context. *â€“end note*]

1.2.5 `OneWayExecutor` requirements

The `OneWayExecutor` requirements specify requirements for executors which create execution agents. A type `X` satisfies the `OneWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

[*Note:* `OneWayExecutor`s provides fire-and-forget semantics without a channel for awaiting the completion of a submitted function object and obtaining its result. *â€“end note*]

In the Table below, `x` denotes a (possibly const) executor object of type `X` and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
`x.execute(f)`	`void`	Creates an execution agent which invokes `DECAY_COPY( std::forward<F>(f))()` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `execute`. May block pending completion of `DECAY_COPY( std::forward<F>(f))()`. The invocation of `execute` synchronizes with (C++Std [intro.multithread]) the invocation of `f`. `execute` shall not propagate any exception thrown by `DECAY_COPY( std::forward<F>(f))()` or any other function submitted to the executor. [*Note:* The treatment of exceptions thrown by one-way submitted functions and the forward

progress guarantee of execution agents created by one-way execution functions are specific to the concrete executor type. *â€“end note.*]

1.2.6 `TwoWayExecutor` requirements

The `TwoWayExecutor` requirements specify requirements for executors which create execution agents with a channel for awaiting the completion of a submitted function object and obtaining its result.

A type `X` satisfies the `TwoWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below, `x` denotes a (possibly const) executor object of type `X`, `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))()` and where `decay_t<F>` satisfies the `MoveConstructible` requirements, and `R` denotes the type of the expression `DECAY_COPY(std::forward<F>(f))()`.

Expression	Return Type	Operational semantics
`x.twoway_execute(f)`	A type that satisfies the `Future` requirements for the value type `R`.	Creates an execution agent which invokes `DECAY_COPY( std::forward<F>(f))()` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `twoway_execute`. May block pending completion of `DECAY_COPY( std::forward<F>(f))()`. The invocation of `twoway_execute` synchronizes with (C++Std [intro.multithread]) the invocation of `f`. Stores the result of `DECAY_COPY( std::forward<F>(f))()`, or any exception thrown by `DECAY_COPY( std::forward<F>(f))()`, in the associated shared state of the resulting `Future`.

1.2.7 `ThenExecutor` requirements

The `ThenExecutor` requirements specify requirements for executors which create execution agents whose initiation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type `X` satisfies the `ThenExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `fut` denotes a future object satisfying the `Future` requirements,
- `val` denotes any object stored in `futâ€™s` associated shared state when it becomes nonexceptionally ready,
- `e` denotes the object stored in `futâ€™s` associated shared state when it becomes exceptionally ready,
- `NORMAL` denotes the expression `DECAY_COPY(std::forward<F>(f))(val)` if `futâ€™s` value type is non-`void` and `DECAY_COPY(std::forward<F>(f))()` if `futâ€™s` value type is `void`,
- `EXCEPTIONAL` denotes the expression `DECAY_COPY(std::forward<F>(f))(exception_arg, e)`,
- `f` denotes a function object of type `F&&` callable as `NORMAL` or `EXCEPTIONAL` and where `decay_t<F>` satisfies the `MoveConstructible` requirements,
- and `R` denotes the type of the expression `NORMAL`.

Expression	Return Type	Operational semantics
		When `fut` becomes nonexceptionally ready, and if `NORMAL` is a well-formed expression, creates an execution agent which invokes `NORMAL` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `then_execute`. Otherwise, when `fut` becomes exceptionally ready, if `EXCEPTIONAL` is a well-formed expression, creates an execution agent which invokes `EXCEPTIONAL` at most once, with the call to `DECAY_COPY` being evaluated in the thread that called `then_execute`. If `NORMAL` and `EXCEPTIONAL` are both well-formed expressions, `decltype(EXCEPTIONAL)` shall be convertible to `R`.

`x.then_execute(f, fut)`	A type that satisfies the `Future` requirements for the value type `R`.	If `NORMAL` is not a well-formed expression and `EXCEPTIONAL` is a well-formed expression, `decltype(EXCEPTIONAL)` shall be convertible to `decltype(val)`. If neither `NORMAL` nor `EXCEPTIONAL` are well-formed expresions, the invocation of `then_execute` shall be ill-formed. May block pending completion of `NORMAL` or `EXCEPTIONAL`. The invocation of `then_execute` synchronizes with (C++Std [intro.multithread]) the invocation of `f`. Stores the result of either the `NORMAL` or `EXCEPTIONAL` expression, or any exception thrown by either, in the associated shared state of the resulting `Future`. Otherwise, stores either `val` or `e` in the associated shared state of the resulting `Future`.

1.2.8 `BulkOneWayExecutor` requirements

The `BulkOneWayExecutor` requirements specify requirements for executors which create groups of execution agents in bulk from a single execution function, without a channel for awaiting the completion of the submitted function object invocations and obtaining their result. [*Note:* That is, the executor provides fire-and-forget semantics. *â€“end note*]

A type `X` satisfies the `BulkOneWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly const) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
`x.bulk_execute(f, n, sf)`	`void`	Invokes `sf()` at most once on an unspecified execution agent to produce the value `s`. Creates a group of execution agents of shape `n` which invokes `DECAY_COPY( std::forward<F>(f))(i, s)` at most once for each value of `i` in the range `[0,n)`, with the call to `DECAY_COPY` being evaluated in the thread that called `bulk_execute`. May block pending completion of one or more calls to `DECAY_COPY( std::forward<F>(f))(i, s)`. The invocation of `bulk_execute` synchronizes with (C++Std [intro.multithread]) the invocations of `f`. `bulk_execute` shall not propagate any exception thrown by `DECAY_COPY( std::forward<F>(f))(i, s)` or any other function submitted to the executor. [*Note:* The treatment of exceptions thrown by bulk one-way submitted functions and the forward progress guarantee of execution agents created by one-way execution functions is specific to the concrete executor type. *â€“end note.*]

1.2.9 `BulkTwoWayExecutor` requirements

The `BulkTwoWayExecutor` requirements specify requirements for executors which create groups of execution agents in bulk from a single execution function with a channel for awaiting the completion of a submitted function object invoked by those execution agents and obtaining its result.

A type `X` satisfies the `BulkTwoWayExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly const) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- if `R` is non-void,
  - `r` denotes an object whose type is `R`,
  - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, r, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.
- if `R` is void,
  - `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
`x.bulk_twoway_execute(f, n, rf, sf)`	A type that satisfies the `Future` requirements for the value type `R`.	If `R` is non-void, invokes `rf()` at most once on an unspecified execution agent to produce the value `r`. Invokes `sf()` at most once on an unspecified execution agent to produce the value `s`. Creates a group of execution agents of shape `n` which invokes `DECAY_COPY( std::forward<F>(f))(i, r, s)` if `R` is non-void, and otherwise invokes `DECAY_COPY( std::forward<F>(f))(i, s)`, at most once for each value of `i` in the range `[0,n)`, with the call to `DECAY_COPY` being evaluated in the thread that called `bulk_twoway_execute`. May block pending completion of one or more invocations of `f`. The invocation of `bulk_twoway_execute` synchronizes with (C++Std [intro.multithread]) the invocations of `f`. Once all invocations of `f` finish execution, stores `r`, or any exception thrown by an invocation of `f`, in the associated shared state of the resulting `Future`.

1.2.10 `BulkThenExecutor` requirements

The `BulkThenExecutor` requirements specify requirements for executors which create execution agents whose initiation is predicated on the readiness of a specified future, and which provide a channel for awaiting the completion of the submitted function object and obtaining its result.

A type `X` satisfies the `BulkThenExecutor` requirements if it satisfies the general requirements on executors, as well as the requirements in the table below.

In the Table below,

- `x` denotes a (possibly const) executor object of type `X`,
- `n` denotes a shape object whose type is `executor_shape_t<X>`,
- `fut` denotes a future object satisfying the Future requirements,
- `val` denotes any object stored in `fut`â€™s associated shared state when it becomes nonexceptionally ready,
- `e` denotes the object stored in `fut`â€™s associated shared state when it becomes exceptionally ready,
- `rf` denotes a `CopyConstructible` function object with zero arguments whose result type is `R`,
- `sf` denotes a `CopyConstructible` function object with zero arguments whose result type is `S`,
- `i` denotes a (possibly const) object whose type is `executor_index_t<X>`,
- `s` denotes an object whose type is `S`,
- if `R` is non-void,
  - `r` denotes an object whose type is `R`,
  - `NORMAL` denotes the expression `DECAY_COPY(std::forward<F>(f))(i, val, r, s)`,
  - `EXCEPTIONAL` denotes the expression `DECAY_COPY(std::forward<F>(f))(i, exception_arg, e, r, s)`,
- if `R` is void,
  - `NORMAL` denotes the expression `DECAY_COPY(std::forward<F>(f))(i, val, s)`,
  - `EXCEPTIONAL` denotes the expression `DECAY_COPY(std::forward<F>(f))(i, exception_arg, e, s)`,
- and `f` denotes a function object of type `F&&` callable as `DECAY_COPY(std::forward<F>(f))(i, fut, s)` and where `decay_t<F>` satisfies the `MoveConstructible` requirements.

Expression	Return Type	Operational semantics
		If `R` is non-void, invokes `rf()` at most once on an unspecified execution agent to produce the value `r`. Invokes `sf()` at most

		once on an unspecified execution agent to produce the value `s`. When `fut` becomes nonexceptionally ready, and if `NORMAL` is a well-formed expression, creates a group of execution agents of shape `n` which invokes `NORMAL` at most once for each value of `i` in the range `[0,n)`, with the call to `DECAY_COPY` being evaluated in the thread that called `bulk_then_execute`. Otherwise, when `fut` becomes exceptionally ready, and if `EXCEPTIONAL` is a well-formed expression, creates a group of execution agents of shape `n` which invokes `EXCEPTIONAL` at most once for each value of `i` in the range `[0,n)`, with the call to `DECAY_COPY` being evaluated in the thread that called `bulk_then_execute`. If neither `NORMAL` nor `EXCEPTIONAL` are well-formed expressions, the invocation of `bulk_then_execute` shall be ill-formed May block pending completion of one or more invocations of `f`. The invocation of `bulk_then_execute` synchronizes with (C++Std [intro.multithread]) the invocations of `f`. Once all invocations of `f` finish execution, stores `r`, or any exception thrown by an invocation of `f`, in the associated shared state of the resulting `Future`. Otherwise, when `fut` becomes exceptionally ready, and if `EXCEPTIONAL` is an ill-formed expression, stores `e` in the associated shared state of the resulting `Future`.
`x.bulk_then_execute(f, n, fut, rf, sf)`	A type that satisfies the `Future` requirements for the value type `R`.	

1.3 Executor customization points

Executor customization points are functions which adapt an executorâ€™s properties. Executor customization points enable uniform use of executors in generic contexts.

When an executor customization point named *NAME* invokes a free execution function of the same name, overload resolution is performed in a context that includes the declaration `void` *NAME*`(auto&... args) = delete;`, where `sizeof...(args)` is the arity of the free execution function. This context also does not include a declaration of the executor customization point.

[*Note:* This provision allows executor customization points to call the executorâ€™s free, non-member execution function of the same name without recursion. *â€“end note*]

Whenever `std::experimental::executors_v1::execution::`*NAME*`(`*ARGS*`)` is a valid expression, that expression satisfies the syntactic requirements for the free execution function named *NAME* with arity `sizeof...(`*ARGS*`)` with that free execution functionâ€™s semantics.

1.3.1 `require`

```
namespace {
  constexpr unspecified require = unspecified;
}
```

The name `require` denotes a customization point. The effect of the expression `std::experimental::executors_v1::execution::require(E, P0, Pn...)` for some expressions `E` and `P0`, and where `Pn...` represents `N` expressions (where `N` is 0 or more), is equivalent to:

- If `N == 0`, `P0::is_requirable` is true, and the expression `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>> == decay_t<decltype(P0)>::value()` is a well-formed constant expression with value `true`, `E`.
- If `N == 0`, `P0::is_requirable` is true, and the expression `(E).require(P0)` is well-formed, `(E).require(P0)`.
- If `N == 0`, `P0::is_requirable` is true, and the expression `require(E, P0)` is well-formed, `require(E, P0)`.
- If `N > 0` and the expression `std::experimental::executors_v1::execution::require( std::experimental::executors_v1::execution::require(E, P0), Pn...)` is well formed, `std::experimental::executors_v1::execution::require( std::experimental::executors_v1::execution::require(E, P0), Pn...)`.
- Otherwise, `std::experimental::executors_v1::execution::require(E, P0, Pn...)` is ill-formed.

1.3.2 `prefer`

```
namespace {
```

```
  constexpr unspecified prefer = unspecified;
}
```

The name `prefer` denotes a customization point. The effect of the expression `std::experimental::executors_v1::execution::prefer(E, P0, Pn...)` for some expressions `E` and `P0`, and where `Pn...` represents `N` expressions (where `N` is 0 or more), is equivalent to:

- If `N == 0`, `P0::is_preferable` is true, and the expression `decay_t<decltype(P0)>::template static_query_v<decay_t<decltype(E)>> == decay_t<decltype(P0)>::value()` is a well-formed constant expression with value `true`, `E`.
- If `N == 0`, `P0::is_preferable` is true, and the expression `(E).require(P0)` is well-formed, `(E).require(P0)`.
- If `N == 0`, `P0::is_preferable` is true, and the expression `prefer(E, P0)` is well-formed, `prefer(E, P0)`.
- If `N == 0` and `P0::is_preferable` is true, `E`.
- If `N > 0` and the expression `std::experimental::executors_v1::execution::prefer( std::experimental::executors_v1::execution::prefer(E, P0), Pn...)` is well formed, `std::experimental::executors_v1::execution::prefer( std::experimental::executors_v1::execution::prefer(E, P0), Pn...)`.
- Otherwise, `std::experimental::executors_v1::execution::require(E, P0, Pn...)` is ill-formed.

1.3.3 query

```
namespace {
  constexpr unspecified query = unspecified;
}
```

The name `query` denotes a customization point. The effect of the expression `std::experimental::executors_v1::execution::query(E, P)` for some expressions `E` and `P` is equivalent to:

- If the expression `decay_t<decltype(P)>::template static_query_v<decay_t<decltype(E)>>` is a well-formed constant expression, `decay_t<decltype(P)>::template static_query_v<decay_t<decltype(E)>>`.
- If the expression `(E).query(P)` is well-formed, `(E).query(P)`.
- If the expression `query(E, P)` is well-formed, `query(E, P)`.
- Otherwise, `std::experimental::executors_v1::execution::query(E, P)` is ill-formed.

1.3.4 Customization point type traits

```
template<class Executor, class... Properties> struct can_require;
template<class Executor, class... Properties> struct can_prefer;
template<class Executor, class Property> struct can_query;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a UnaryTypeTrait (C++Std [meta.rqmts]) with a BaseCharacteristic of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
`template<class T> struct can_require`	The expression `std::experimental::executors_v1::execution::require( declval<const Executor>(), declval<Properties>()...)` is well formed.	`T` is a complete type.
`template<class T> struct can_prefer`	The expression `std::experimental::executors_v1::execution::prefer( declval<const Executor>(), declval<Properties>()...)` is well formed.	`T` is a complete type.
`template<class T> struct can_query`	The expression `std::experimental::executors_v1::execution::query( declval<const Executor>(), declval<Property>())` is well formed.	`T` is a complete type.

1.4 Executor properties

1.4.1 In general

An executorâ€™s behavior in generic contexts is determined by a set of executor properties, and each executor property imposes certain requirements on the executor.

Given an existing executor, a related executor with different properties may be created by calling the `require` member or non-member functions. These functions behave according the table below. In the table below, `x` denotes a (possibly const) executor object of type `X`, and `p` denotes a (possibly const) property object.

[*Note:* As a general design note properties which define a mutually exclusive pair, that describe an enabled or non-enabled behaviour follow the convention of having the same property name for both with the `not_` prefix to the property for the non-enabled behaviour. *â€“end note*]

Expression	Comments
`x.require(p)` `require(x,p)`	Returns an executor object with the requested property `p` added to the set. All other properties of the returned executor are identical to those of `x`, except where those properties are described below as being mutually exclusive to `p`. In this case, the mutually exclusive properties are implicitly removed from the set associated with the returned executor. The expression is ill formed if an executor is unable to add the requested property.

The current value of an executorâ€™s properties can be queried by calling the `query` function. This function behaves according the table below. In the table below, `x` denotes a (possibly const) executor object of type `X`, and `p` denotes a (possibly const) property object.

Expression	Comments
`x.query(p)`	Returns the current value of the requested property `p`. The expression is ill formed if an executor is unable to return the requested property.

1.4.2 Requirements on properties

A property type `P` shall provide:

- A nested constant expression named `is_requirable` of type `bool`, usable as `P::is_requirable`.
- A nested constant expression named `is_preferable` of type `bool`, usable as `P::is_preferable`.

[*Note:* These constants are used to determine whether the property can be used with the `require` and `prefer` customization points, respectively. *â€“end note*]

A property type `P` may provide a nested type `polymorphic_query_result_type` that satisfies the `DefaultConstructible`, `CopyConstructible` and `Destructible` requirements. [*Note:* When present, this type allows the property to be used with the polymorphic `executor` wrapper. *â€“end note*]

A property type `P` may provide:

- A nested variable template `static_query_v`, usable as `P::static_query_v<Executor>`. This may be conditionally present.
- A member function `value()`.

If both `static_query_v` and `value()` are present, they shall return the same type and this type shall satisfy the `EqualityComparable` requirements.

[*Note:* These are used to determine whether a `require` call would result in an identity transformation. *â€“end note*]

1.4.3 Query-only properties

1.4.3.1 Associated execution context property

```
struct context_t
{
  static constexpr bool is_requirable = false;
  static constexpr bool is_preferable = false;

  using polymorphic_query_result_type = any; // TODO: alternatively consider void*, or simply omitting the type.

  template<class Executor>
    static constexpr decltype(auto) static_query_v
      = Executor::query(context_t());
};
```

The `context_t` property can be used only with `query`, which returns the **execution context** associated with the executor.

An execution context is a program object that represents a specific collection of execution resources and the **execution agents** that exist within those resources. Execution agents are units of execution, and a 1-to-1 mapping exists between an execution agent and an invocation of a callable function object submitted via the executor.

The value returned from `execution::query(e, context_t)`, where `e` is an executor, shall not change between calls.

1.4.4 Interface-changing properties

1.4.4.1 Directionality properties

```
struct oneway_t;
struct twoway_t;
struct then_t;
```

The directionality properties conform to the following specification:

```
struct S
{
  static constexpr bool is_requirable = true;
  static constexpr bool is_preferable = false;

  using polymorphic_query_result_type = bool;

  template<class Executor>
    static constexpr bool static_query_v
      = see-below;

  static constexpr bool value() const { return true; }
};
```

Property	Requirements
`oneway_t`	The executor type satisfies the `OneWayExecutor` or `BulkOneWayExecutor` requirements.
`twoway_t`	The executor type satisfies the `TwoWayExecutor` or `BulkTwoWayExecutor` requirements.
`then_t`	The executor type satisfies the `ThenExecutor` or `BulkThenExecutor` requirements.

`S::static_query_v<Executor>` is true if and only if `Executor` fulfills sâ€™s requirements.

The `oneway_t`, `twoway_t` and `then_t` properties are not mutually exclusive.

1.4.4.1.1 `twoway_t` customization points

In addition to conforming to the above specification, the `twoway_t` property provides the following customization:

```
struct twoway_t
{
  template<class Executor>
    friend see-below require(Executor ex, twoway_t);
};
```

This customization point returns an executor that satisfies the `twoway_t` requirements by adapting the native functionality of an executor that does not satisfy the `twoway_t` requirements.

Returns: A value `e1` of type `E1` that holds a copy of `ex`. `E1` has member functions `require` and `query` that forward to the corresponding members of the copy of `ex`, if present. For some type `T`, the type yielded by `executor_future_t<E1, T>` is `executor_future_t<Executor, T>` if `then_t::static_query_v<Executor>` is true; otherwise, it is `std::experimental::future<T>`. `e1` has the same properties as `ex`, except for the addition of the `twoway_t` property. Additional `twoway_t` requirements are satisfied as follows:

- If `single_t::static_query_v<Executor> && then_t::static_query_v<Executor>` is true, then `E1` has member function `twoway_execute` implemented in terms of `then_execute`. Otherwise, if `single_t::static_query_v<Executor> && oneway_t::static_query_v<Executor> && adaptable_blocking_t::static_query_v<Executor>` is true, then `E1` has member function `twoway_execute` implemented in terms of `execute`.
- If `bulk_t::static_query_v<Executor> && then_t::static_query_v<Executor>` is true, then `E1` has member function `bulk_twoway_execute` implemented in terms of `bulk_then_execute`. Otherwise, if `bulk_t::static_query_v<Executor> && oneway_t::static_query_v<Executor> && adaptable_blocking_t::static_query_v<Executor>` is true, then `E1` has member function `bulk_twoway_execute` implemented in terms of `bulk_execute`.

Remarks: This function shall not participate in overload resolution unless `twoway_t::template static_query_v<Executor>` is false and `then_t::static_query_v<Executor> || (oneway_t::static_query_v<Executor> && adaptable_blocking_t::static_query_v<Executor>)` is true.

1.4.4.2 Cardinality properties

```
struct single_t;
struct bulk_t;
```

The cardinality properties conform to the following specification:

```
struct S
{
  static constexpr bool is_requirable = true;
  static constexpr bool is_preferable = false;

  using polymorphic_query_result_type = bool;
```

```
  template<class Executor>
    static constexpr bool static_query_v
      = see-below;

  static constexpr bool value() const { return true; }
};
```

Property	Requirements
`single_t`	The executor type satisfies the `OneWayExecutor`, `TwoWayExecutor`, or `ThenExecutor` requirements.
`bulk_t`	The executor type satisfies the `BulkOneWayExecutor`, `BulkTwoWayExecutor`, or `BulkThenExecutor` requirements.

`S::static_query_v<Executor>` is true if and only if `Executor` fulfills sâ€™s requirements.

The `single_t` and `bulk_t` properties are not mutually exclusive.

1.4.4.2.1 `single_t` customization points

In addition to conforming to the above specification, the `single_t` property provides the following customization:

```
struct single_t
{
  template<class Executor>
    friend see-below require(Executor ex, single_t);
};
```

This customization point returns an executor that satisfies the `single_t` requirements by adapting the native functionality of an executor that does not satisfy the `single_t` requirements.

Returns: A value `e1` of type `E1` that holds a copy of `ex`. `E1` has member functions `require` and `query` that forward to the corresponding members of the copy of `ex`, if present. For some type `T`, the type yielded by `executor_future_t<E1, T>` is `executor_future_t<Executor, T>` if `twoway_t::static_query_v<Executor> || then_t::static_query_v<Executor>` is true; otherwise, it is `std::experimental::future<T>`. `e1` has the same properties as `ex`, except for the addition of the `single_t` property. Additional `single_t` requirements are satisfied as follows:

- If `oneway_t::static_query_v<Executor> && bulk_t::static_query_v<Executor>` is true, then `E1` has member function `execute` implemented in terms of `bulk_execute`.
- If `twoway_t::static_query_v<Executor> && bulk_t::static_query_v<Executor>` is true, then `E1` has member function `twoway_execute` implemented in terms of `bulk_twoway_execute`.
- If `then_t::static_query_v<Executor> && bulk_t::static_query_v<Executor>` is true, then `E1` has member function `then_execute` implemented in terms of `bulk_then_execute`.

Remarks: This function shall not participate in overload resolution unless `single_t::static_query_v<Executor>` is false and `bulk_t::static_query_v<Executor>` is true.

1.4.4.2.2 `bulk_t` customization points

In addition to conforming to the above specification, the `bulk_t` property provides the following customization:

```
struct bulk_t
{
  template<class Executor>
    friend see-below require(Executor ex, bulk_t);
};
```

If the executor has the `oneway_t` property, this customization returns an adapter that implements the bulk property and its requirements.

```
template<class Executor>
  friend see-below require(Executor ex, bulk_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. If `Executor` satisfies the `OneWayExecutor` requirements, `E1` shall satisfy the `OneWayExecutor` requirements by providing member functions `require`, `query`, and `execute` that forward to the corresponding member functions of the copy of `ex`, if present, and `E1` shall satisfy the `BulkExecutor` requirements by implementing `bulk_execute` in terms of `execute`. If `Executor` also satisfies the `TwoWayExecutor` requirements, `E1` shall satisfy the `TwoWayExecutor` requirements by providing the member function `twoway_execute` that forwards to the corresponding member function of the copy of `ex`, if present, and `E1` shall satisfy the `BulkTwoWayExecutor` requirements by implementing `bulk_twoway_execute` in terms of `bulk_execute`. `e1` has the same executor properties as `ex`, except for the addition of the `bulk_t` property.

Remarks: This function shall not participate in overload resolution unless `is_bulk_oneway_executor_v<Executor> || is_bulk_twoway_executor_v<Executor>` is false, and `is_oneway_executor_v<Executor>` is true.

1.4.5 Behavioral properties

Behavioral properties define a set of mutually-exclusive nested properties describing executor behavior.

Unless otherwise specified, behavioral property types `S`, their nested property types `S::N`*i*, and nested property objects `S::n`*i* conform to the following specification:

```
struct S
{
  static constexpr bool is_requirable = false;
  static constexpr bool is_preferable = false;
  using polymorphic_query_result_type = S;

  template<class Executor>
    static constexpr auto static_query_v
      = see-below;

  template<class Executor>
  friend constexpr S query(const Executor& ex, const Property& p) noexcept(see-below);

  friend constexpr bool operator==(const S& a, const S& b);
  friend constexpr bool operator!=(const S& a, const S& b) { return !operator==(a, b); }

  constexpr S();

  struct N1
  {
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;
    using polymorphic_query_result_type = S;

    template<class Executor>
      static constexpr auto static_query_v
        = see-below;

    static constexpr S value() { return S(N1()); }
  };

  static constexpr n1;

  constexpr S(const N1);

  ...

  struct NN
  {
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;
    using polymorphic_query_result_type = S;

    template<class Executor>
      static constexpr auto static_query_v
        = see-below;

    static constexpr S value() { return S(NN()); }
  };

  static constexpr nN;

  constexpr S(const NN);
};
```

Queries for the value of an executorâ€™s behavioral property shall not change between calls unless the executor is assigned another executor with a different value of that behavioral property.

`S()` and `S(S::E`*i*`())` are all distinct values of `S`. [*Note:* This means they compare unequal. *â€“end note.*]

The value returned from `execution::query(e1, p1)` and a subsequent call `execution::query(e1, p1)`, where

- `p1` is an instance of `S` or `S::E`*i*, and
- `e2` is the result of `execution::require(e1, p2)` or `execution::prefer(e1, p2)`,

shall compare equal unless

- `p2` is an instance of `S::E`*i*, and
- `p1` and `p2` are different types.

The value of the expression `S::N1::static_query_v<Executor>` is

- `Executor::query(S::N1())`, if that expression is a well-formed expression;
- ill-formed if `declval<Executor>().query(S::N1())` is well-formed;
- ill-formed if `can_query_v<Executor,S::N`*i*`>` is `true` for any `1 <` *i* `<= N`;
- otherwise `S::N1()`.

[*Note:* These rules automatically enable the `S::N1` property by default for executors which do not provide a `query` function for properties

`S::N`*i*. â€“*end note*]

The value of the expression `S::N`*i*`::static_query_v<Executor>`, for all `1 <` *i* `<= N`, is

- `Executor::query(S::N`*i*`())`, if that expression is a well-formed constant expression;
- otherwise ill-formed.

The value of the expression `S::static_query_v<Executor>` is

- `Executor::query(S())`, if that expression is a well-formed constant expression;
- otherwise, ill-formed if `declval<Executor>().query(S())` is well-formed;
- otherwise, `S::N`*i*`::static_query_v<Executor>` for the least *i* `<= N` for which this expression is a well-formed constant expression;
- otherwise ill-formed.

[*Note:* These rules automatically enable the `S::N1` property by default for executors which do not provide a `query` function for properties `S` or `S::N`*i*. â€“*end note*]

Let *k* be the least value of *i* for which `can_query_v<Executor,S::N`*i*`>` is true, if such a value of *i* exists.

```
template<class Executor>
  friend constexpr S query(const Executor& ex, const Property& p) noexcept(noexcept(execution::query(ex, std::declval<const S::Nk>())));
```

Returns: `execution::query(ex, S::N`*k*`())`.

Remarks: This function shall not participate in overload resolution unless `is_same_v<Property,S> && can_query_v<Executor,S::N`*i*`>` is true for at least one `S::N`*i*`.

```
bool operator==(const S& a, const S& b);
```

Returns: `true` if `a` and `b` were constructed from the same constructor; `false`, otherwise.

1.4.5.1 Blocking properties

The `blocking_t` property describes what guarantees executors provide about the blocking behavior of their execution functions.

`blocking_t` provides nested property types and objects as described below.

Nested Property Type	Nested Property Object Name	Requirements
`blocking_t::possibly_t`	`blocking_t::possibly`	A call to an executorâ€™s execution function may block pending completion of one or more of the execution agents created by that execution function.
`blocking_t::always_t`	`blocking_t::always`	A call to an executorâ€™s execution function shall block until completion of all execution agents created by that execution function.
`blocking_t::never_t`	`blocking_t::never`	A call to an executorâ€™s execution function shall not block pending completion of the execution agents created by that execution function.

1.4.5.1.1 `blocking_t::always_t` customization points

In addition to conforming to the above specification, the `blocking_t::always_t` property provides the following customization:

```
struct always_t
{
  template<class Executor>
    friend see-below require(Executor ex, blocking_t::always_t);
};
```

If the executor has the `blocking_adaptation_t::allowed_t` property, this customization uses an adapter to implement the `blocking_t::always_t` property.

```
template<class Executor>
  friend see-below require(Executor ex, blocking_t::always_t);
```

Returns: A value `e1` of type `E1` that holds a copy of `ex`. If `Executor` satisfies the `OneWayExecutor` requirements, `E1` shall satisfy the `OneWayExecutor` requirements by providing member functions `require`, `query`, and `execute` that forward to the corresponding member

functions of the copy of ex. If Executor satisfies the TwoWayExecutor requirements, E1 shall satisfy the TwoWayExecutor requirements by providing member functions require, query, and twoway_execute that forward to the corresponding member functions of the copy of ex. If Executor satisfies the BulkOneWayExecutor requirements, E1 shall satisfy the BulkOneWayExecutor requirements by providing member functions require, query, and bulk_execute that forward to the corresponding member functions of the copy of ex. If Executor satisfies the BulkTwoWayExecutor requirements, E1 shall satisfy the BulkTwoWayExecutor requirements by providing member functions require, query, and bulk_twoway_execute that forward to the corresponding member functions of the copy of ex. In addition, E1 provides an overload of require such that e1.require(blocking.always) returns a copy of e1, an overload of query such that e1.query(blocking) returns blocking.always, and all functions execute, twoway_execute, bulk_execute, and bulk_twoway_execute shall block the calling thread until the submitted functions have finished execution. e1 has the same executor properties as ex, except for the addition of the blocking_t::always_t property, and removal of blocking_t::never_t and blocking_t::possibly_t properties if present.

Remarks: This function shall not participate in overload resolution unless blocking_adaptation_t::static_query_v<Executor> is blocking_adaptation.allowed.

1.4.5.2 Properties to indicate if blocking and directionality may be adapted

The blocking_adaptation_t property allows or disallows blocking or directionality adaptation via execution::require.

blocking_adaptation_t provides nested property types and objects as described below.

Nested Property Type	Nested Property Object Name	Requirements
blocking_adaptation_t::disallowed_t	blocking_adaptation::disallowed	The require customization point may not adapt the executor to add the twoway_t or blocking_t::always_t properties.
blocking_adaptation_t::allowed_t	blocking_adaptation::allowed	The require customization point may adapt the executor to add the twoway_t or blocking_t::always_t properties.

[*Note:* The twoway_t property is included here as the require customization pointâ€™s twoway_t adaptation is specified in terms of std::experimental::future, and that template supports blocking wait operations. *â€“end note*]

1.4.5.2.1 blocking_adaptation_t::allowed_t customization points

In addition to conforming to the above specification, the blocking_adaptation_t::allowed_t property provides the following customization:

```
struct allowed_t
{
  template<class Executor>
    friend see-below require(Executor ex, blocking_adaptation_t::allowed_t);
};
```

This customization uses an adapter to implement the blocking_adaptation_t::allowed_t property.

```
template<class Executor>
  friend see-below require(Executor ex, blocking_adaptation_t::allowed_t);
```

Returns: A value e1 of type E1 that holds a copy of ex. If Executor satisfies the OneWayExecutor requirements, E1 shall satisfy the OneWayExecutor requirements by providing member functions require, query, and execute that forward to the corresponding member functions of the copy of ex. If Executor satisfies the TwoWayExecutor requirements, E1 shall satisfy the TwoWayExecutor requirements by providing member functions require, query, and twoway_execute that forward to the corresponding member functions of the copy of ex. If Executor satisfies the BulkOneWayExecutor requirements, E1 shall satisfy the BulkOneWayExecutor requirements by providing member functions require, query, and bulk_execute that forward to the corresponding member functions of the copy of ex. If Executor satisfies the BulkTwoWayExecutor requirements, E1 shall satisfy the BulkTwoWayExecutor requirements by providing member functions require, query, and bulk_twoway_execute that forward to the corresponding member functions of the copy of ex. In addition, blocking_adaptation_t::static_query_v<E1> is blocking_adaptation.allowed, and e1.require(blocking_adaptation.disallowed) yields a copy of ex. e1 has the same executor properties as ex, except for the addition of the blocking_adaptation_t::allowed_t property.

1.4.5.3 Properties to indicate if submitted tasks represent continuations

The relationship_t property allows users of executors to indicate that submitted tasks represent continuations.

relationship_t provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
		Function objects

`relationship_t::fork_t`	`relationship_t::fork`	submitted through the executor do not represent continuations of the caller.
`relationship_t::continuation_t`	`relationship_t::continuation`	Function objects submitted through the executor represent continuations of the caller. If the caller is a lightweight execution agent managed by the executor or its associated execution context, the execution of the submitted function object may be deferred until the caller completes.

1.4.5.4 Properties to indicate likely task submission in the future

The `outstanding_work_t` property allows users of executors to indicate that task submission is likely in the future.

`outstanding_work_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
`outstanding_work_t::untracked_t`	`outstanding_work::untracked`	The existence of the executor object does not indicate any likely future submission of a function object.
`outstanding_work_t::tracked_t`	`outstanding_work::tracked`	The existence of the executor object represents an indication of likely future submission of a function object. The executor or its associated execution context may choose to maintain execution resources in anticipation of this submission.

[*Note:* The `outstanding_work_t::tracked_t` and `outstanding_work_t::untracked_t` properties are used to communicate to the associated execution context intended future work submission on the executor. The intended effect of the properties is the behavior of execution contextâ€™s facilities for awaiting outstanding work; specifically whether it considers the existance of the executor object with the `outstanding_work_t::tracked_t` property enabled outstanding work when deciding what to wait on. However this will be largely defined by the execution context implementation. It is intended that the execution context will define its wait facilities and on-destruction behaviour and provide an interface for querying this. An initial work towards this is included in P0737r0. *â€“end note*]

1.4.5.5 Properties for bulk execution guarantees

Bulk execution guarantee properties communicate the forward progress and ordering guarantees of execution agents with respect to other agents within the same bulk submission.

`bulk_guarantee_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
`bulk_guarantee_t::unsequenced_t`	`bulk_guarantee_t::unsequenced`	Execution agents within the same bulk execution may be parallelized and vectorized.
`bulk_guarantee_t::sequenced_t`	`bulk_guarantee_t::sequenced`	Execution agents within the same bulk execution may not be parallelized.
`bulk_guarantee_t::parallel_t`	`bulk_guarantee_t::parallel`	Execution agents within the same bulk execution

may be parallelized.

Execution agents created by executors with the `bulk_guarantee_t::unsequenced_t` property may execute in an unordered fashion. Any such agents executing in the same thread of execution are unsequenced with respect to each other. [*Note:* This means that multiple execution agents may be interleaved on a single thread of execution, which overrides the usual guarantee from [intro.execution] that function executions do not interleave with one another. *â€“end note*]

Execution agents created by executors with the `bulk_guarantee_t::sequenced_t` property execute in sequence in lexicographic order of their indices.

Execution agents created by executors with the `bulk_guarantee_t::parallel_t` property execute with a parallel forward progress guarantee. Any such agents executing in the same thread of execution are indeterminately sequenced with respect to each other. [*Note:* It is the callerâ€™s responsibility to ensure that the invocation does not introduce data races or deadlocks. *â€“end note*]

[*Editorial note:* The descriptions of these properties were ported from [algorithms.parallel.user]. The intention is that a future standard will specify execution policy behavior in terms of the fundamental properties of their associated executors. We did not include the accompanying code examples from [algorithms.parallel.user] because the examples seem easier to understand when illustrated by `std::for_each`. *â€“end editorial note*]

1.4.5.6 Properties for mapping of execution on to threads

The `mapping_t` property describes what guarantees executors provide about the mapping of execution agents onto threads of execution.

`mapping_t` provides nested property types and objects as indicated below.

Nested Property Type	Nested Property Object Name	Requirements
`mapping_t::thread_t`	`mapping::thread`	Execution agents created by the executor are mapped onto threads of execution.
`mapping_t::new_thread_t`	`mapping::new_thread`	Each execution agent created by the executor is mapped onto a new thread of execution.
`mapping_t::other_t`	`mapping::other`	Mapping of each execution agent created by the executor is implementation-defined.

[*Note:* A mapping of an execution agent onto a thread of execution implies the agent executes as-if on a `std::thread`. Therefore, the facilities provided by `std::thread`, such as thread-local storage, are available. `mapping_t::new_thread_t` provides stronger guarantees, in particular that thread-local storage will not be shared between execution agents. *â€“end note*]

1.4.6 Properties for customizing memory allocation

```
template <typename ProtoAllocator>
struct allocator_t;
```

The `allocator_t` property conforms to the following specification:

```
template <typename ProtoAllocator>
struct allocator_t
{
    static constexpr bool is_requirable = true;
    static constexpr bool is_preferable = true;

    template<class Executor>
    static constexpr auto static_query_v
      = Executor::query(allocator_t);

    template <typename OtherProtoAllocator>
    allocator_t<OtherProtoAllocator> operator()(const OtherProtoAllocator &a) const {
        return allocator_t<OtherProtoAllocator>{a};
    }

    static constexpr ProtoAllocator value() const {
      return a_; // exposition only
    }

private:
    ProtoAllocator a_; // exposition only
};
```

Property	Requirements

`allocator_t<ProtoAllocator>`	Result of `allocator_t<void>::operator(OtherProtoAllocator)`. The executor type satisfies the `OneWayExecutor`, `TwoWayExecutor`, or `ThenExecutor` requirements. The executor implementation shall use the encapsulated allocator to allocate any memory required to store the submitted function object.
`allocator_t<void>`	Specialisation of `allocator_t<ProtoAllocator>`. The executor type satisfies the `OneWayExecutor`, `TwoWayExecutor`, or `ThenExecutor` requirements. The executor implementation shall use an implementation defined default allocator to allocate any memory required to store the submitted function object.

Remarks: `operator(OtherProtoAllocator)` and `value()` shall not participate in overload resolution unless `ProtoAllocator` is `void`.

Postconditions: `alloc.value()` returns `a`, where `alloc` is the result of `allocator(a)`.

[*Note:* Where the `allocator_t` is queryable, it must be accepted as both `allocator_t<ProtoAllocator>` and `allocator_t<void>`. *â€“end note*]

[*Note:* As the `allocator_t<ProtoAllocator>` property enapsulates a value which can be set and queried, it is required to be implemented such that it is callable with the `OtherProtoAllocator` parameter where the customization points accepts the result of `allocator_t<void>::operator(OtherProtoAllocator)`; `allocator_t<OtherProtoAllocator>` and is passable as an instance where the customization points accept an instance of `allocator_t<void>`. *â€“end note*]

[*Note:* It is permitted for an allocator provided via `allocator_t<void>::operator(OtherProtoAllocator)` property to be the same type as the default allocator provided by the implementation. *â€“end note*]

1.5 Executor type traits

1.5.1 Determining that a type satisfies executor type requirements

```
template<class T> struct is_oneway_executor;
template<class T> struct is_twoway_executor;
template<class T> struct is_then_executor;
template<class T> struct is_bulk_oneway_executor;
template<class T> struct is_bulk_twoway_executor;
template<class T> struct is_bulk_then_executor;
```

This sub-clause contains templates that may be used to query the properties of a type at compile time. Each of these templates is a UnaryTypeTrait (C++Std [meta.rqmts]) with a BaseCharacteristic of `true_type` if the corresponding condition is true, otherwise `false_type`.

Template	Condition	Preconditions
`template<class T>` `struct is_oneway_executor`	`T` meets the syntactic requirements for `OneWayExecutor`.	`T` is a complete type.
`template<class T>` `struct is_twoway_executor`	`T` meets the syntactic requirements for `TwoWayExecutor`.	`T` is a complete type.
`template<class T>` `struct is_then_executor`	`T` meets the syntactic requirements for `ThenExecutor`.	`T` is a complete type.
`template<class T>` `struct is_bulk_oneway_executor`	`T` meets the syntactic requirements for `BulkOneWayExecutor`.	`T` is a complete type.
`template<class T>` `struct is_bulk_twoway_executor`	`T` meets the syntactic requirements for `BulkTwoWayExecutor`.	`T` is a complete type.
`template<class T>` `struct is_bulk_then_executor`	`T` meets the syntactic requirements for `BulkThenExecutor`.	`T` is a complete type.

1.5.2 Associated future type

```
template<class Executor, class T>
struct executor_future
{
  using type = decltype(execution::require(declval<const Executor&>(), execution::twoway).twoway_execute(declval<T(*)()>()));
};
```

1.5.3 Associated shape type

```
template<class Executor>
struct executor_shape
{
  private:
    // exposition only
    template<class T>
    using helper = typename T::shape_type;

  public:
    using type = std::experimental::detected_or_t<
```

```
      size_t, helper, decltype(execution::require(declval<const Executor&>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "shape type must be an integral type");
};
```

1.5.4 Associated index type

```
template<class Executor>
struct executor_index
{
  private:
    // exposition only
    template<class T>
    using helper = typename T::index_type;

  public:
    using type = std::experimental::detected_or_t<
      executor_shape_t<Executor>, helper, decltype(execution::require(declval<const Executor&>(), execution::bulk))
    >;

    // exposition only
    static_assert(std::is_integral_v<type>, "index type must be an integral type");
};
```

1.6 Polymorphic executor wrappers

[*Commentary: The polymorphic executor wrapper class has been written such that it could be separated from the rest of the proposal if necessary.*]

In several places in this section the operation `CONTAINS_PROPERTY(p, pn)` is used. All such uses mean `std::disjunction_v<std::is_same<p, pn>...>`.

In several places in this section the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is used. All such uses mean the first type `P` in the parameter pack `pn` for which `std::is_convertible_v<p, P>` is `true`. If no such type `P` exists, the operation `FIND_CONVERTIBLE_PROPERTY(p, pn)` is ill-formed.

1.6.1 Class `bad_executor`

An exception of type `bad_executor` is thrown by `executor` member functions `execute`, `twoway_execute`, `bulk_execute`, and `bulk_twoway_execute` when the executor object has no target.

```
class bad_executor : public exception
{
public:
  // constructor:
  bad_executor() noexcept;
};
```

1.6.1.1 `bad_executor` constructors

```
bad_executor() noexcept;
```

Effects: Constructs a `bad_executor` object.

Postconditions: `what()` returns an implementation-defined NTBS.

1.6.2 Class template `executor`

The `executor` class template provides a polymorphic wrapper for executor types.

```
template <class... SupportableProperties>
class executor
{
public:
  // construct / copy / destroy:

  executor() noexcept;
  executor(nullptr_t) noexcept;
  executor(const executor& e) noexcept;
  executor(executor&& e) noexcept;
  template<class Executor> executor(Executor e);
  template<class... OtherSupportableProperties> executor(executor<OtherSupportableProperties...> e);
  template<class... OtherSupportableProperties> executor(executor<OtherSupportableProperties...> e) = delete;

  executor& operator=(const executor& e) noexcept;
  executor& operator=(executor&& e) noexcept;
```

```
  executor& operator=(nullptr_t) noexcept;
  template<class Executor> executor& operator=(Executor e);

  ~executor();

  // executor modifiers:

  void swap(executor& other) noexcept;

  // executor operations:

  template <class Property>
  executor require(Property) const;

  template <class Property>
  typename Property::polymorphic_query_result_type query(Property) const;

  template<class Function>
    void execute(Function&& f) const;

  template<class Function>
    std::experimental::future<result_of_t<decay_t<Function>()>>
      twoway_execute(Function&& f) const

  template<class Function, class SharedFactory>
    void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

  template<class Function, class ResultFactory, class SharedFactory>
    std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
      bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;

  // executor capacity:

  explicit operator bool() const noexcept;

  // executor target access:

  const type_info& target_type() const noexcept;
  template<class Executor> Executor* target() noexcept;
  template<class Executor> const Executor* target() const noexcept;
};

// executor comparisons:

template <class... SupportableProperties>
bool operator==(const executor<SupportableProperties...>& a, const executor<SupportableProperties...>& b) noexcept;
template <class... SupportableProperties>
bool operator==(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template <class... SupportableProperties>
bool operator==(nullptr_t, const executor<SupportableProperties...>& e) noexcept;
template <class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& a, const executor<SupportableProperties...>& b) noexcept;
template <class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template <class... SupportableProperties>
bool operator!=(nullptr_t, const executor<SupportableProperties...>& e) noexcept;

// executor specialized algorithms:

template <class... SupportableProperties>
void swap(executor<SupportableProperties...>& a, executor<SupportableProperties...>& b) noexcept;

template <class Property, class... SupportableProperties>
executor prefer(const executor<SupportableProperties>& e, Property p);
```

The `executor` class satisfies the general requirements on executors.

[*Note:* To meet the `noexcept` requirements for executor copy constructors and move constructors, implementations may share a target between two or more `executor` objects. *â€“end note*]

Each property type in the `SupportableProperties...` pack shall provide a nested type `polymorphic_query_result_type`.

The *target* is the executor object that is held by the wrapper.

1.6.2.1 `executor` constructors

```
executor() noexcept;
```

Postconditions: `!*this`.

```
executor(nullptr_t) noexcept;
```

Postconditions: `!*this`.

```
executor(const executor& e) noexcept;
```

Postconditions: `!*this` if `!e`; otherwise, `*this` targets `e.target()` or a copy of `e.target()`.

```
executor(executor&& e) noexcept;
```

Effects: If `!e`, `*this` has no target; otherwise, moves `e.target()` or move-constructs the target of `e` into the target of `*this`, leaving `e` in a valid state with an unspecified value.

```
template<class Executor> executor(Executor e);
```

Remarks: This function shall not participate in overload resolution unless: * `can_require_v<Executor, P>`, if `P::is_requireable`, where `P` is each property in `SupportableProperties...`. * `can_prefer_v<Executor, P>`, if `P::is_preferable`, where `P` is each property in `SupportableProperties...`. * and `can_query_v<Executor, P>`, where `P` is each property in `SupportableProperties...`.

Effects: * `*this` targets a copy of `e5` initialized with `std::move(e5)`, where: * If `CONTAINS_PROPERTY(execution::single_t, SupportableProperties)`, `e1` is the result of `execution::require(e, execution::single_t)`, otherwise `e1` is `e`, * If `CONTAINS_PROPERTY(execution::bulk_t, SupportableProperties)`, `e2` is the result of `execution::require(e, execution::bulk_t)`, otherwise `e2` is `e1` * If `CONTAINS_PROPERTY(execution::oneway_t, SupportableProperties)`, `e3` is the result of `execution::require(e, execution::oneway_t)`, otherwise `e3` is `e2` * If `CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties)`, `e4` is the result of `execution::require(e, execution::twoway_t)`, otherwise `e4` is `e3`. * * If `CONTAINS_PROPERTY(execution::then_t, SupportableProperties)`, `e5` is the result of `execution::require(e, execution::then_t)`, otherwise `e5` is `e4`.

```
template<class... OtherSupportableProperties> executor(executor<OtherSupportableProperties...> e);
```

Remarks: This function shall not participate in overload resolution unless: * `CONTAINS_PROPERTY(p, OtherSupportableProperties)`, where `p` is each property in `SupportableProperties...`.

Effects: `*this` targets a copy of `e` initialized with `std::move(e)`.

```
template<class... OtherSupportableProperties> executor(executor<OtherSupportableProperties...> e) = delete;
```

Remarks: This function shall not participate in overload resolution unless `CONTAINS_PROPERTY(p, OtherSupportableProperties)` is `false` for some property `p` in `SupportableProperties...`.

1.6.2.2 `executor` assignment

```
executor& operator=(const executor& e) noexcept;
```

Effects: `executor(e).swap(*this)`.

Returns: `*this`.

```
executor& operator=(executor&& e) noexcept;
```

Effects: Replaces the target of `*this` with the target of `e`, leaving `e` in a valid state with an unspecified value.

Returns: `*this`.

```
executor& operator=(nullptr_t) noexcept;
```

Effects: `executor(nullptr).swap(*this)`.

Returns: `*this`.

```
template<class Executor> executor& operator=(Executor e);
```

Requires: As for `template<class Executor> executor(Executor e)`.

Effects: `executor(std::move(e)).swap(*this)`.

Returns: `*this`.

1.6.2.3 `executor` destructor

```
~executor();
```

Effects: If `*this != nullptr`, releases shared ownership of, or destroys, the target of `*this`.

1.6.2.4 `executor` modifiers

```
void swap(executor& other) noexcept;
```

Effects: Interchanges the targets of `*this` and `other`.

1.6.2.5 `executor` operations

```
template <class Property>
executor require(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)::is_requirable` is well-formed and has the value `true`.

Returns: A polymorphic wrapper whose target is the result of `execution::require(e, p)`, where `e` is the target object of `*this`.

```
template <class Property>
typename Property::polymorphic_query_result_type query(Property p) const;
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)` is well-formed.

Returns: If `executor::query(e, p)` is well-formed, `static_cast<Property::polymorphic_query_result_type>(executor::query(e, p))`, where `e` is the target object of `*this`. Otherwise, `Property::polymorphic_query_result_type{}`.

```
template<class Function>
  void execute(Function&& f) const;
```

Remarks: This function shall not participate in overload resolution unless: * `CONTAINS_PROPERTY(execution::oneway_t, SupportableProperties)`, * and `CONTAINS_PROPERTY(execution::single_t, SupportableProperties)`.

Effects: Performs `e.execute(f2)`, where:

- `e` is the target object of `*this`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when called as `f2()`, performs `f1()`.

```
template<class Function>
  std::experimental::future<result_of_t<decay_t<Function>()>>
    twoway_execute(Function&& f) const
```

Remarks: This function shall not participate in overload resolution unless: * `CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties)`, * and `CONTAINS_PROPERTY(execution::single_t, SupportableProperties)`.

Effects: Performs `e.twoway_execute(f2)`, where:

- `e` is the target object of `*this`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when called as `f2()`, performs `f1()`.

Returns: A future, whose shared state is made ready when the future returned by `e.twoway_execute(f2)` is made ready, containing the result of `f1()` or any exception thrown by `f1()`. [*Note:* `e2.twoway_execute(f2)` may return any future type that satisfies the Future requirements, and not necessarily `std::experimental::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that objectâ€™s `then()` member function. When invoked, this continuation stores the result in the outer futureâ€™s associated shared and makes that shared state ready. *â€“end note*]

```
template<class Function, class SharedFactory>
  void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Remarks: This function shall not participate in overload resolution unless: * `CONTAINS_PROPERTY(execution::oneway_t, SupportableProperties)`, * and `CONTAINS_PROPERTY(execution::bulk_t, SupportableProperties)`.

Effects: Performs `e.bulk_execute(f2, n, sf2)`, where:

- `e` is the target object of `*this`;
- `sf1` is the result of `DECAY_COPY(std::forward<SharedFactory>(sf))`;
- `sf2` is a function object of unspecified type that, when called as `sf2()`, performs `sf1()`;
- `s1` is the result of `sf1()`;
- `s2` is the result of `sf2()`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- `f2` is a function object of unspecified type that, when called as `f2(i, s2)`, performs `f1(i, s1)`, where `i` is a value of type `size_t`.

```
template<class Function, class ResultFactory, class SharedFactory>
  std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
    void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Remarks: This function shall not participate in overload resolution unless: * `CONTAINS_PROPERTY(execution::twoway_t, SupportableProperties)`, * and `CONTAINS_PROPERTY(execution::bulk_t, SupportableProperties)`.

Effects: Performs `e.bulk_twoway_execute(f2, n, rf2, sf2)`, where:

- `e` is the target object of `*this`;
- `rf1` is the result of `DECAY_COPY(std::forward<ResultFactory>(rf))`;
- `rf2` is a function object of unspecified type that, when called as `rf2()`, performs `rf1()`;

- `sf1` is the result of `DECAY_COPY(std::forward<SharedFactory>(rf))`;
- `sf2` is a function object of unspecified type that, when called as `sf2()`, performs `sf1()`;
- if `decltype(rf1())` is non-void, `r1` is the result of `rf1()`;
- if `decltype(rf2())` is non-void, `r2` is the result of `rf2()`;
- `s1` is the result of `sf1()`;
- `s2` is the result of `sf2()`;
- `f1` is the result of `DECAY_COPY(std::forward<Function>(f))`;
- if `decltype(rf1())` is non-void and `decltype(rf2())` is non-void, `f2` is a function object of unspecified type that, when called as `f2(i, r2, s2)`, performs `f1(i, r1, s1)`, where `i` is a value of type `size_t`.
- if `decltype(rf1())` is non-void and `decltype(rf2())` is void, `f2` is a function object of unspecified type that, when called as `f2(i, s2)`, performs `f1(i, r1, s1)`, where `i` is a value of type `size_t`.
- if `decltype(rf1())` is void and `decltype(rf2())` is non-void, `f2` is a function object of unspecified type that, when called as `f2(i, r2, s2)`, performs `f1(i, s1)`, where `i` is a value of type `size_t`.
- if `decltype(rf1())` is void and `decltype(rf2())` is void, `f2` is a function object of unspecified type that, when called as `f2(i, s2)`, performs `f1(i, s1)`, where `i` is a value of type `size_t`.

Returns: A future, whose shared state is made ready when the future returned by `e.bulk_twoway_execute(f2, n, rf2, sf2)` is made ready, containing the result in `r1` (if `decltype(rf1())` is non-void) or any exception thrown by an invocation`f1`. [*Note:* `e.bulk_twoway_execute(f2)` may return any future type that satisfies the Future requirements, and not necessarily `std::experimental::future`. One possible implementation approach is for the polymorphic wrapper to attach a continuation to the inner future via that objectâ€™s `then()` member function. When invoked, this continuation stores the result in the outer futureâ€™s associated shared and makes that shared state ready. *â€“end note*]

1.6.2.6 `executor` capacity

```
explicit operator bool() const noexcept;
```

Returns: `true` if `*this` has a target, otherwise `false`.

1.6.2.7 `executor` target access

```
const type_info& target_type() const noexcept;
```

Returns: If `*this` has a target of type `T`, `typeid(T)`; otherwise, `typeid(void)`.

```
template<class Executor> Executor* target() noexcept;
template<class Executor> const Executor* target() const noexcept;
```

Returns: If `target_type() == typeid(Executor)` a pointer to the stored executor target; otherwise a null pointer value.

1.6.2.8 `executor` comparisons

```
template<class... SupportableProperties>
bool operator==(const executor<SupportableProperties...>& a, const executor<SupportableProperties...>& b) noexcept;
```

Returns:

- `true` if `!a` and `!b`;
- `true` if `a` and `b` share a target;
- `true` if `e` and `f` are the same type and `e == f`, where `e` is the target of `a` and `f` is the target of `b`;
- otherwise `false`.

```
template<class... SupportableProperties>
bool operator==(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator==(nullptr_t, const executor<SupportableProperties...>& e) noexcept;
```

Returns: `!e`.

```
template<class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& a, const executor<SupportableProperties...>& b) noexcept;
```

Returns: `!(a == b)`.

```
template<class... SupportableProperties>
bool operator!=(const executor<SupportableProperties...>& e, nullptr_t) noexcept;
template<class... SupportableProperties>
bool operator!=(nullptr_t, const executor<SupportableProperties...>& e) noexcept;
```

Returns: `(bool) e`.

1.6.2.9 `executor` specialized algorithms

```
template<class... SupportableProperties>
```

```
void swap(executor<SupportableProperties...>& a, executor<SupportableProperties...>& b) noexcept;
```

Effects: `a.swap(b)`.

```
template <class Property, class... SupportableProperties>
executor prefer(const executor<SupportableProperties...>& e, Property p);
```

Remarks: This function shall not participate in overload resolution unless `FIND_CONVERTIBLE_PROPERTY(Property, SupportableProperties)::is_preferable` is well-formed and has the value `true`.

Returns: A polymorphic wrapper whose target is the result of `execution::prefer(e, p)`, where `e` is the target object of `*this`.

1.6.3 Struct `prefer_only`

The `prefer_only` struct is a property adapter that disables the `is_requirable` value.

[*Example:*

Consider a generic function that performs some task immediately if it can, and otherwise asynchronously in the background.

```
template<class Executor>
void do_async_work(
    Executor ex,
    Callback callback)
{
  if (try_work() == done)
  {
    // Work completed immediately, invoke callback.
    execution::require(ex,
        execution::single,
        execution::oneway,
      ).execute(callback);
  }
  else
  {
    // Perform work in background. Track outstanding work.
    start_background_work(
        execution::prefer(ex,
          execution::outstanding_work.tracked),
        callback);
  }
}
```

This function can be used with an inline executor which is defined as follows:

```
struct inline_executor
{
  constexpr bool operator==(const inline_executor&) const noexcept
  {
    return true;
  }

  constexpr bool operator!=(const inline_executor&) const noexcept
  {
    return false;
  }

  template<class Function> void execute(Function f) const noexcept
  {
    f();
  }
};
```

as, in the case of an unsupported property, the `execution::prefer` call will fall back to an identity operation.

The polymorphic `executor` wrapper should be able to simply swap in, so that we could change `do_async_work` to the non-template function:

```
void do_async_work(
    executor<
      execution::single,
      execution::oneway,
      execution::outstanding_work_t::tracked_t> ex,
    std::function<void()> callback)
{
  if (try_work() == done)
  {
    // Work completed immediately, invoke callback.
    execution::require(ex,
        execution::single,
        execution::oneway,
      ).execute(callback);
  }
```

```
  else
  {
    // Perform work in background. Track outstanding work.
    start_background_work(
        execution::prefer(ex,
          execution::outstanding_work.tracked),
        callback);
  }
}
```

with no change in behavior or semantics.

However, if we simply specify `execution::outstanding_work.tracked` in the `executor` template parameter list, we will get a compile error. This is because the `executor` template doesnâ€™t know that `execution::outstanding_work.tracked` is intended for use with `prefer` only. At the point of construction from an `inline_executor` called `ex`, `executor` will try to instantiate implementation templates that perform the ill-formed `execution::require(ex, execution::outstanding_work.tracked)`.

The `prefer_only` adapter addresses this by turning off the `is_requirable` attribute for a specific property. It would be used in the above example as follows:

```
void do_async_work(
    executor<
      execution::single,
      execution::oneway,
      prefer_only<execution::outstanding_work_t::tracked_t>> ex,
    std::function<void()> callback)
{
  ...
}
```

â€“ end example]

```
template<class InnerProperty>
struct prefer_only
{
  InnerProperty property;

  static constexpr bool is_requirable = false;
  static constexpr bool is_preferable = InnerProperty::is_preferable;

  using polymorphic_query_result_type = see-below; // not always defined

  template<class Executor>
    static constexpr auto static_query_v = see-below; // not always defined

  constexpr prefer_only(const InnerProperty& p);

  constexpr auto value() const
    noexcept(noexcept(std::declval<const InnerProperty>().value()))
      -> decltype(std::declval<const InnerProperty>().value());

  template<class Executor, class Property>
  friend auto prefer(Executor ex, const Property& p)
    noexcept(noexcept(execution::prefer(std::move(ex), std::declval<const InnerProperty>())))
      -> decltype(execution::prefer(std::move(ex), std::declval<const InnerProperty>()));

  template<class Executor, class Property>
  friend constexpr auto query(const Executor& ex, const Property& p)
    noexcept(noexcept(execution::query(ex, std::declval<const InnerProperty>())))
      -> decltype(execution::query(ex, std::declval<const InnerProperty>()));
};
```

If `InnerProperty::polymorphic_query_result_type` is valid and denotes a type, the template instantiation `prefer_only<InnerProperty>` defines a nested type `polymorphic_query_result_type` as a synonym for `InnerProperty::polymorphic_query_result_type`.

If `InnerProperty::static_query_v` is a variable template and `InnerProperty::static_query_v<E>` is well formed for some executor type `E`, the template instantiation `prefer_only<InnerProperty>` defines a nested variable template `static_query_v` as a synonym for `InnerProperty::static_query_v`.

```
constexpr prefer_only(const InnerProperty& p);
```

Effects: Initializes `property` with `p`.

```
constexpr auto value() const
  noexcept(noexcept(std::declval<const InnerProperty>().value()))
    -> decltype(std::declval<const InnerProperty>().value());
```

Returns: `property.value()`.

Remarks: Shall not participate in overload resolution unless the expression `property.value()` is well-formed.

```
template<class Executor, class Property>
```

```
friend auto prefer(Executor ex, const Property& p)
  noexcept(noexcept(execution::prefer(std::move(ex), std::declval<const InnerProperty>())))
    -> decltype(execution::prefer(std::move(ex), std::declval<const InnerProperty>()));
```

Returns: `execution::prefer(std::move(ex), p.property)`.

Remarks: Shall not participate in overload resolution unless `std::is_same_v<Property, prefer_only>` is `true`, and the expression `execution::prefer(std::move(ex), p.property)` is well-formed.

```
template<class Executor, class Property>
friend constexpr auto query(const Executor& ex, const Property& p)
  noexcept(noexcept(execution::query(ex, std::declval<const InnerProperty>())))
    -> decltype(execution::query(ex, std::declval<const InnerProperty>()));
```

Returns: `execution::query(ex, p.property)`.

Remarks: Shall not participate in overload resolution unless `std::is_same_v<Property, prefer_only>` is `true`, and the expression `execution::query(ex, p.property)` is well-formed.

1.7 Thread pools

Thread pools create execution agents which execute on threads without incurring the overhead of thread creation and destruction whenever such agents are needed.

1.7.1 Header `<thread_pool>` synopsis

```
namespace std {
namespace experimental {
inline namespace executors_v1 {

  class static_thread_pool;

} // inline namespace executors_v1
} // namespace experimental
} // namespace std
```

1.7.2 Class `static_thread_pool`

`static_thread_pool` is a statically-sized thread pool which may be explicitly grown via thread attachment. The `static_thread_pool` is expected to be created with the use case clearly in mind with the number of threads known by the creator. As a result, no default constructor is considered correct for arbitrary use cases and `static_thread_pool` does not support any form of automatic resizing.

`static_thread_pool` presents an effectively unbounded input queue and the execution functions of `static_thread_pool`â€™s associated executors do not block on this input queue.

[*Note:* Because `static_thread_pool` represents work as parallel execution agents, situations which require concurrent execution properties are not guaranteed correctness. *â€“end note.*]

```
class static_thread_pool
{
  public:
    using executor_type = see-below;

    // construction/destruction
    explicit static_thread_pool(std::size_t num_threads);

    // nocopy
    static_thread_pool(const static_thread_pool&) = delete;
    static_thread_pool& operator=(const static_thread_pool&) = delete;

    // stop accepting incoming work and wait for work to drain
    ~static_thread_pool();

    // attach current thread to the thread pools list of worker threads
    void attach();

    // signal all work to complete
    void stop();

    // wait for all threads in the thread pool to complete
    void wait();

    // placeholder for a general approach to getting executors from
    // standard contexts.
    executor_type executor() noexcept;
};
```

For an object of type `static_thread_pool`, *outstanding work* is defined as the sum of:

- the number of existing executor objects associated with the `static_thread_pool` for which the `execution::outstanding_work.tracked` property is established;
- the number of function objects that have been added to the `static_thread_pool` via the `static_thread_pool` executor, but not yet executed; and
- the number of function objects that are currently being executed by the `static_thread_pool`.

The `static_thread_pool` member functions `executor`, `attach`, `wait`, and `stop`, and the associated executorsâ€™ copy constructors and member functions, do not introduce data races as a result of concurrent calls to those functions from different threads of execution.

A `static_thread_pool`â€™s threads execute execution agents created via its associated executors with forward progress guarantee delegation. [*Note:* Forward progress is delegated to an execution agent for its lifetime. Because `static_thread_pool` guarantees only parallel forward progress to execution agents created via its executors, forward progress delegation does not apply to execution agents which have not yet started executing their first execution step. *â€“end note*]

1.7.2.1 Types

```
using executor_type = see-below;
```

An executor type conforming to the specification for `static_thread_pool` executor types described below.

1.7.2.2 Construction and destruction

```
static_thread_pool(std::size_t num_threads);
```

Effects: Constructs a `static_thread_pool` object with `num_threads` threads of execution, as if by creating objects of type `std::thread`.

```
~static_thread_pool();
```

Effects: Destroys an object of class `static_thread_pool`. Performs `stop()` followed by `wait()`.

1.7.2.3 Worker management

```
void attach();
```

Effects: Adds the calling thread to the pool such that this thread is used to execute submitted function objects. [*Note:* Threads created during thread pool construction, or previously attached to the pool, will continue to be used for function object execution. *â€“end note*] Blocks the calling thread until signalled to complete by `stop()` or `wait()`, and then blocks until all the threads created during `static_thread_pool` object construction have completed. (NAMING: a possible alternate name for this function is `join()`.)

```
void stop();
```

Effects: Signals the threads in the pool to complete as soon as possible. If a thread is currently executing a function object, the thread will exit only after completion of that function object. The call to `stop()` returns without waiting for the threads to complete. Subsequent calls to attach complete immediately.

```
void wait();
```

Effects: If not already stopped, signals the threads in the pool to complete once the outstanding work is `0`. Blocks the calling thread (C++Std [defns.block]) until all threads in the pool have completed, without executing submitted function objects in the calling thread. Subsequent calls to attach complete immediately.

Synchronization: The completion of each thread in the pool synchronizes with (C++Std [intro.multithread]) the corresponding successful `wait()` return.

1.7.2.4 Executor creation

```
executor_type executor() noexcept;
```

Returns: An executor that may be used to submit function objects to the thread pool. The returned executor has the following properties already established:

- `execution::oneway`
- `execution::twoway`
- `execution::then`
- `execution::single`
- `execution::bulk`
- `execution::blocking.possibly`
- `execution::continuation.no`
- `execution::outstanding_work.untracked`
- `execution::allocator`
- `execution::allocator(std::allocator<void>())`

1.7.3 `static_thread_pool` executor types

All executor types accessible through `static_thread_pool::executor()`, and subsequent calls to the member function `require`, conform to the following specification.

```
class C
{
  public:
    // types:

    typedef std::size_t shape_type;
    typedef std::size_t index_type;

    // construct / copy / destroy:

    C(const C& other) noexcept;
    C(C&& other) noexcept;

    C& operator=(const C& other) noexcept;
    C& operator=(C&& other) noexcept;

    // executor operations:

    see-below require(execution::blocking_t::never_t) const;
    see-below require(execution::blocking_t::possibly_t) const;
    see-below require(execution::blocking_t::always_t) const;
    see-below require(execution::relationship_t::continuation_t) const;
    see-below require(execution::relationship_t::fork_t) const;
    see-below require(execution::outstanding_work_t::tracked_t) const;
    see-below require(execution::outstanding_work_t::untracked_t) const;
    see-below require(const execution::allocator_t<void>& a) const;
    template<class ProtoAllocator>
    see-below require(const execution::allocator_t<ProtoAllocator>& a) const;

    static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t::parallel_t) const;
    static constexpr execution::mapping_t query(execution::mapping_t::thread_t) const;
    execution::blocking_t query(execution::blocking_t) const;
    execution::relationship_t query(execution::relationship_t) const;
    execution::outstanding_work_t query(execution::outstanding_work_t) const;
    see-below query(execution::context_t) const noexcept;
    see-below query(execution::allocator_t<void>) const noexcept;
    template<class ProtoAllocator>
    see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;

    bool running_in_this_thread() const noexcept;

    template<class Function>
      void execute(Function&& f) const;

    template<class Function>
      std::experimental::future<result_of_t<decay_t<Function>()>>
        twoway_execute(Function&& f) const

    template<class Function, class Future>
      std::experimental::future<result_of_t<decay_t<Function>(decay_t<Future>)>>
        then_execute(Function&& f, Future&& pred) const;

    template<class Function, class SharedFactory>
      void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;

    template<class Function, class ResultFactory, class SharedFactory>
      std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
        void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
};

bool operator==(const C& a, const C& b) noexcept;
bool operator!=(const C& a, const C& b) noexcept;
```

`C` is a type satisfying the `OneWayExecutor`, `TwoWayExecutor`, `BulkOneWayExecutor`, and `BulkTwoWayExecutor` requirements. Objects of type `C` are associated with a `static_thread_pool`.

1.7.3.1 Constructors

```
C(const C& other) noexcept;
```

Postconditions: `*this == other`.

```
C(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

1.7.3.2 Assignment

```
C& operator=(const C& other) noexcept;
```

Postconditions: `*this == other`.

Returns: `*this`.

```
C& operator=(C&& other) noexcept;
```

Postconditions: `*this` is equal to the prior value of `other`.

Returns: `*this`.

1.7.3.3 Operations

```
see-below require(execution::blocking_t::never_t) const;
see-below require(execution::blocking_t::possibly_t) const;
see-below require(execution::blocking_t::always_t) const;
see-below require(execution::relationship_t::continuation_t) const;
see-below require(execution::relationship_t::fork_t) const;
see-below require(execution::outstanding_work_t::tracked_t) const;
see-below require(execution::outstanding_work_t::untracked_t) const;
```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, and having the requested property established. When the requested property is part of a group that is defined as a mutually exclusive set, any other properties in the group are removed from the returned executor object. All other properties of the returned executor object are identical to those of `*this`.

```
see-below require(const execution::allocator_t<void>& a) const;
```

Returns: `require(execution::allocator(x))`, where `x` is an implementation-defined default allocator.

```
template<class ProtoAllocator>
  see-below require(const execution::allocator_t<ProtoAllocator>& a) const;
```

Returns: An executor object of an unspecified type conforming to these specifications, associated with the same thread pool as `*this`, with the `execution::allocator_t<ProtoAllocator>` property established such that allocation and deallocation associated with function submission will be performed using a copy of `a.alloc`. All other properties of the returned executor object are identical to those of `*this`.

```
static constexpr execution::bulk_guarantee_t query(execution::bulk_guarantee_t) const;
```

Returns: `execution::bulk_guarantee.parallel`

```
static constexpr execution::mapping_t query(execution::mapping_t) const;
```

Returns: `execution::mapping.thread`.

```
execution::blocking_t query(execution::blocking_t) const;
execution::relationship_t query(execution::relationship_t) const;
execution::outstanding_work_t query(execution::outstanding_work_t) const;
```

Returns: The value of the given property of `*this`.

```
static_thread_pool& query(execution::context_t) const noexcept;
```

Returns: A reference to the associated `static_thread_pool` object.

```
see-below query(execution::allocator_t<void>) const noexcept;
see-below query(execution::allocator_t<ProtoAllocator>) const noexcept;
```

Returns: The allocator object associated with the executor, with type and value as either previously established by the `execution::allocator_t<ProtoAllocator>` property or the implementation defined default allocator established by the `execution::allocator_t<void>` property.

```
bool running_in_this_thread() const noexcept;
```

Returns: `true` if the current thread of execution is a thread that was created by or attached to the associated `static_thread_pool` object.

```
template<class Function>
  void execute(Function&& f) const;
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `OneWayExecutor` requirements and the properties established for `*this`. If the submitted function `f` exits via an exception, the `static_thread_pool` calls `std::terminate()`.

```
template<class Function>
  std::experimental::future<result_of_t<decay_t<Function>()>>
    twoway_execute(Function&& f) const
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `TwoWayExecutor` requirements and the properties established for `*this`.

Returns: A future with behavior as specified by the `TwoWayExecutor` requirements.

```
template<class Function, class Future>
  std::experimental::future<result_of_t<decay_t<Function>(decay_t<Future>)>>
    then_execute(Function&& f, Future&& pred) const
```

Effects: Submits the function `f` for execution on the `static_thread_pool` according to the `ThenExecutor` requirements and the properties established for `*this`.

Returns: A future with behavior as specified by the `ThenExecutor` requirements.

```
template<class Function, class SharedFactory>
  void bulk_execute(Function&& f, size_t n, SharedFactory&& sf) const;
```

Effects: Submits the function `f` for bulk execution on the `static_thread_pool` according to the `BulkOneWayExecutor` requirements and the properties established for `*this`. If the submitted function `f` exits via an exception, the `static_thread_pool` calls `std::terminate()`.

```
template<class Function, class ResultFactory, class SharedFactory>
  std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
    void bulk_twoway_execute(Function&& f, size_t n, ResultFactory&& rf, SharedFactory&& sf) const;
```

Effects: Submits the function `f` for bulk execution on the `static_thread_pool` according to the `BulkTwoWayExecutor` requirements and the properties established for `*this`.

Returns: A future with behavior as specified by the `BulkTwoWayExecutor` requirements.

```
template<class Function, class Future, class ResultFactory, class SharedFactory>
  std::experimental::future<result_of_t<decay_t<ResultFactory>()>>
    void bulk_then_execute(Function&& f, size_t n, Future&& pred, ResultFactory&& rf, SharedFactory&& sf) const;
```

Effects: Submits the function `f` for bulk execution on the `static_thread_pool` according to the `BulkThenExecutor` requirements and the properties established for `*this`.

Returns: A future with behavior as specified by the `BulkThenExecutor` requirements.

1.7.3.4 Comparisons

```
bool operator==(const C& a, const C& b) noexcept;
```

Returns: `true` if `&a.query(execution::context) == &b.query(execution::context)` and `a` and `b` have identical properties, otherwise `false`.

```
bool operator!=(const C& a, const C& b) noexcept;
```

Returns: `!(a == b)`.